CONFLUENT

PLATFORM

Quick Start Guide

Local Installation · Topics · Connectors
ksqlDB Streams & Tables · Monitoring

Rajesh Pasham

01
Download & Install
Prerequisites, env setup, Hub install

02
Create Kafka Topics
Control Center UI walkthrough

03
Install Connector
Datagen: pageviews & users mock data

04
ksqlDB Streams
Streams, tables, persistent queries

05
Monitor Consumer Lag
Control Center consumer lag view

06
Stop & Clean Up
Stop services · Destroy local data

What This Quick Start Covers

A local Confluent Platform instance — all components running on your machine

ZooKeeper

Coordination

`localhost:2181`

Apache Kafka

Event broker

`localhost:9092`

Schema Registry

Schema store

`localhost:8081`

Kafka REST

HTTP proxy

`localhost:8082`

Kafka Connect

Connectors

`localhost:8083`

ksqlDB

SQL engine

`localhost:8088`

Control Center

GUI & monitor

`localhost:9021`

```
$ confluent local services start   →   starts ALL components above in order
```

01

Download & Start Confluent Platform

Prerequisites · Environment setup · Installing the Datagen connector · Starting all services

Step 1 — Prerequisites & Download

Get Confluent Platform running locally in under 10 minutes

Prerequisites

🌐 Internet connectivity

Required to download Platform and install connectors from Confluent Hub.

💻 Supported operating system

Linux, macOS, or Windows (WSL2). Check confluent.io/installation for current list.

☕ Java 8 or Java 11

Java 9 and 10 are NOT supported. Verify with: java -version

💾 Disk space

Minimum 2 GB free for Platform binaries and local data directories.

Download Steps

1. **Visit Confluent downloads page**
   Go to confluent.io/download and scroll to 'Download Confluent Platform'
2. **Choose your format**
   Select deb (Debian/Ubuntu), rpm (RHEL/CentOS), tar, or zip archive
3. **Enter email & download**
   Provide your email address and click DOWNLOAD FREE
4. **Decompress the archive**
   Extract to get the bin/, etc/, share/ directory structure

Step 1 — Environment Setup & Start

Set PATH variables, install Datagen connector, launch all services

Environment Variables

```bash
# Set CONFLUENT_HOME
$ export CONFLUENT_HOME=<path-to-confluent>
# Add bin/ to PATH
$ export PATH=$PATH:$CONFLUENT_HOME/bin
$ confluent version
```

Install Datagen Connector

Datagen generates mock streaming data for demos — not for production use.

```bash
$ confluent-hub install --no-prompt \
    confluentinc/kafka-connect-datagen:latest
```

Start All Services

One command starts ZooKeeper, Kafka, Schema Registry, REST Proxy, Connect, ksqlDB, and Control Center.

```bash
$ confluent local services start

Starting Zookeeper ... Zookeeper is [UP]
Starting Kafka ....... Kafka is [UP]
Starting Schema Registry ... [UP]
Starting Kafka REST ........ [UP]
Starting Connect ........... [UP]
Starting KSQL Server ....... [UP]
Starting Control Center .... [UP]

# All 7 components running ✓
```

02

Create Kafka Topics

Use Confluent Control Center at http://localhost:9021 to create pageviews and users topics

Step 2 — Create Kafka Topics via Control Center

Navigate to localhost:9021 → Topics → Add a topic

Topics to Create

Topic: pageviews

Simulates web page view events. Produced by Datagen in Avro format at 100ms intervals.

```
Schema: pageid (STRING) · userid (STRING) · viewtime (LONG)
```

Topic: users

Simulates a user profile table. Produced by Datagen in Avro format at 1000ms intervals.

```
Schema: id (STRING) · gender (STRING) · regionid (STRING) · registertime (LONG)
```

Control Center Navigation

1. **`http://localhost:9021`**
   Open Control Center in your browser. May take 1–2 min to come online after start.
2. **`Click: controlcenter.cluster`**
   Select your local cluster tile from the landing page.
3. **`Nav bar → Topics`**
   Open the topics list from the left navigation panel.
4. **`Click: Add a topic`**
   Opens the topic creation dialog.
5. **`Name: "pageviews" → Create`**
   Enter topic name (case-sensitive) and accept defaults.
6. **`Repeat for "users" topic`**
   Add a second topic with the name 'users'.

03

Install Connector & Generate Sample Data

Configure two Datagen connector instances — one for pageviews, one for users

Step 3 — Configure Datagen Connectors

Two connector instances: datagen-pageviews (100ms) and datagen-users (1000ms)

datagen-pageviews

Produces mock page view events to the pageviews topic every 100ms in Avro format.

Configuration Property	Value
Key converter class	StringConverter
kafka.topic	pageviews
max.interval	100
quickstart	pageviews

bash

```
# Connect > Add connector > DatagenConnector > Next > Launch
```

datagen-users

Produces mock user profiles to the users topic every 1000ms in Avro format.

Configuration Property	Value
Key converter class	StringConverter
kafka.topic	users
max.interval	1000
quickstart	users

bash

```
# Nav: Connect > connect-default > Add connector > Launch
```

04

ksqlDB — Streams, Tables & Queries

Create streams and tables, write persistent queries, filter and join in real-time

Step 4 — Create Stream and Table in ksqlDB

Navigate: Control Center → ksqlDB → Editor → Run query

Stream vs Table — Key Concept

STREAM: an unbounded, append-only sequence of events — like a Kafka topic viewed over time.

TABLE: a materialised view with the latest state per key — like a database table continuously updated by a Kafka topic.

Create pageviews STREAM

Registers the pageviews Kafka topic as a ksqlDB stream. Avro schema is resolved from Schema Registry automatically.

ksqlDB

```
CREATE STREAM pageviews
WITH (
  KAFKA_TOPIC='pageviews',
  VALUE_FORMAT='AVRO'
);
```

Create users TABLE

Registers the users topic as a ksqlDB table. Each record updates the row for that user ID — enabling joins against live data.

ksqlDB

```
CREATE TABLE users
  (id VARCHAR PRIMARY KEY)
WITH (
  KAFKA_TOPIC='users',
  VALUE_FORMAT='AVRO'
);
```

Step 4 — Configure Queries & Run Push Query

Set auto.offset.reset → Earliest, then run your first non-persistent query

Configure Query Properties

auto.offset.reset

Value: Earliest

Tells ksqlDB to read ALL available data from the beginning of the topic, not just new messages after the query starts.

ksql.streams.auto.offset.reset

Value: earliest

Equivalent setting for persistent queries. Without this, queries only see data that arrives after they are created.

Non-Persistent (Push) Query

A non-persistent query streams results live to your browser — results are NOT written back to Kafka. Useful for ad-hoc inspection.

```ksqlDB
SELECT pageid FROM pageviews
  EMIT CHANGES LIMIT 3;
```

Expected output:

```bash
pageid
-------
Page_39
Page_11
Page_74
Limit Reached
```

Step 4 — Persistent Queries (Streams & Tables)

Results are written back to Kafka topics and survive ksqlDB restarts

pageviews_female

Filters pageviews to female users by joining with the users table. Results written to PAGEVIEWS_FEMALE topic.

ksqlDB

```
CREATE STREAM pageviews_female AS
  SELECT users.id AS userid,
    pageid, regionid
  FROM pageviews
  LEFT JOIN users
    ON pageviews.userid = users.id
  WHERE gender = 'FEMALE'
  EMIT CHANGES;
```

pageviews_female_like_89

Further filters female pageviews to regions ending in 8 or 9. Output topic: pageviews_enriched_r8_r9.

ksqlDB

```
CREATE STREAM
pageviews_female_like_89
  WITH (KAFKA_TOPIC=
    'pageviews_enriched_r8_r9',
    VALUE_FORMAT='AVRO')
  AS SELECT * FROM
pageviews_female
  WHERE regionid LIKE '%_8'
    OR regionid LIKE '%_9'
  EMIT CHANGES;
```

pageviews_regions (TABLE)

Counts pageviews per gender+region in a 30-second tumbling window. Results written to PAGEVIEWS_REGIONS topic.

ksqlDB

```
CREATE TABLE pageviews_regions
  WITH (KEY_FORMAT='JSON')
  AS SELECT gender, regionid,
    COUNT(*) AS numusers
  FROM pageviews
  LEFT JOIN users
    ON pageviews.userid = users.id
  WINDOW TUMBLING (SIZE 30 SECOND)
  GROUP BY gender, regionid
  HAVING COUNT(*) > 1
  EMIT CHANGES;
```

Step 4 — Persistent Queries & Stream/Table Browser

View all running queries and browse available streams/tables from the ksqlDB UI

Persistent Queries Tab

Navigate to ksqlDB → "Persistent queries" tab to see all running queries.

CSAS_PAGEVIEWS_FEMALE_0 — STREAM

→ PAGEVIEWS_FEMALE

CSAS_PAGEVIEWS_FEMALE_LIKE_89_1 — STREAM

→ pageviews_enriched_r8_r9

CTAS_PAGEVIEWS_REGIONS_2 — TABLE

→ PAGEVIEWS_REGIONS

Run Queries on Streams & Tables

Streams tab → PAGEVIEWS_FEMALE

Click 'Query stream'

Streaming output displays in the editor. Click Stop to halt.

Tables tab → PAGEVIEWS_REGIONS

Click 'Query table'

Materialised table rows displayed live. Click Stop to halt.

Editor tab → KSQL_PROCESSING_LOG

Click stream name

View the stream's schema including nested data structures.

05

Monitor Consumer Lag

Use Control Center → Consumers to inspect ksqlDB consumer group lag and throughput

Step 5 — Monitor Consumer Lag

Control Center → Consumers → select the ksqlDB consumer group

What Is Consumer Lag?

Consumer lag = the difference between the latest offset in a partition and the last offset committed by the consumer. High lag means the consumer is falling behind the producer.

End Log Offset

The latest offset written to the partition — total messages ever produced.

Current Offset

The last offset successfully consumed and committed by the consumer group.

Lag

End Log Offset − Current Offset. Zero lag = fully caught up consumer.

Consumption Rate

Messages consumed per second — monitor for throughput drops or stalls.

Navigation Path

Control Center (9021) → Consumers tab → _confluent-ksql-default_query_CSAS_PAGEVIEWS_FEMALE_5 → Lag & consumption metrics dashboard

06

Stop & Clean Up

Stop all Confluent Platform services and optionally destroy local data

Step 6 — Stop Confluent Platform

Gracefully stop all services and clean up local data when you are done

Stop Services

Gracefully shuts down all running components in reverse order (Control Center → ksqlDB → Connect → REST → Schema Registry → Kafka → ZooKeeper).

bash

```bash
$ confluent local services stop

Stopping Control Center ....... [DOWN]
Stopping KSQL Server .......... [DOWN]
```

Destroy Local Data

Deletes ALL local data including topics, offsets, schemas, and connector state. Irreversible.

bash

```bash
$ confluent local destroy
# ⚠  All local data will be deleted
```

What Gets Cleaned Up

🗑️ **Kafka topics & offsets**
All topic data and consumer group offsets are deleted from the local broker.

📋 **Schema Registry schemas**
All Avro/JSON/Protobuf schemas registered during the session are removed.

🔌 **Connector state**
Datagen connector configurations and worker state are cleaned up.

📊 **ksqlDB persistent queries**
All persistent stream and table query state and output topics are destroyed.

📁 **Data directories**
The /tmp/confluent.xxx local data directory is fully deleted.

ksqlDB Quick Reference — Cheat Sheet

Key DDL and DML statements used in this Quick Start

DDL — Streams

```
CREATE STREAM s WITH (...);
```

Register a Kafka topic as a stream

```
CREATE STREAM s AS SELECT ...;
```

Persistent derived stream (CSAS)

```
DROP STREAM s DELETE TOPIC;
```

Remove stream and its backing topic

DDL — Tables

```
CREATE TABLE t (id VARCHAR PK) WITH (...);
```

Register topic as a table with key

```
CREATE TABLE t AS SELECT ...;
```

Persistent derived table (CTAS)

```
DROP TABLE t DELETE TOPIC;
```

Remove table and backing topic

DML — Queries

```
SELECT ... EMIT CHANGES;
```

Push query — streams results live

```
SELECT ... LIMIT N;
```

Push query with row limit

```
SELECT ... FROM t WHERE key=...;
```

Pull query — point-in-time lookup

Joins & Windows

```
s LEFT JOIN t ON s.id = t.id
```

Stream-Table join — enrich events

```
WINDOW TUMBLING (SIZE 30 SECOND)
```

Fixed non-overlapping time windows

```
WINDOW SESSION (30 MINUTES)
```

Session windows by user inactivity

Key Takeaways

01

One command starts everything

confluent local services start launches all 7 components — ZooKeeper, Kafka, Schema Registry, REST, Connect, ksqlDB, and Control Center.

02

Control Center is the GUI command hub

Create topics, manage connectors, browse schemas, monitor lag, and inspect ksqlDB streams — all from localhost:9021.

03

Datagen generates instant mock data

Install once via confluent-hub. Two connector instances — pageviews (100ms) and users (1000ms) — populate your topics immediately.

04

ksqlDB makes streaming SQL trivial

CREATE STREAM and CREATE TABLE in seconds. Persistent queries run continuously, writing enriched/filtered results back to Kafka topics.

05

Consumer lag is a first-class metric

The Consumers tab in Control Center shows end offset, current offset, and lag per partition — your primary health indicator.

06

Always destroy local data when done

confluent local destroy cleans up all topics, schemas, connector state, and ksqlDB query state — keeping your machine clean.